Übungsblatt 7: Tracing und Timing-Analyse

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Ziele, Zeit und Voraussetzungen

Sie formulieren einen Messvertrag, rechnen Zeitgrößen und unterscheiden beobachtete Werte von einer allgemeinen Schranke. Vorausgesetzt werden Vorlesungen 1–6, insbesondere Job, Zustände, Pufferbesitz, Shared Memory und Cache. Neue Tracebegriffe werden in Vorlesung 7 eingeführt. Ein Trace ist eine geordnete Aufzeichnung ausgewählter Ereignisse. Ein Messvertrag benennt Anfang, Ende, Quelle, Einheit, Bedingungen und enthaltene Arbeit.

Teil A: vier klausurnahe Papieraufgaben, 32 Punkte und etwa 50 Minuten. Alle Zahlen sind didaktische Annahmen, keine Boardmessungen. Teil B benötigt mit vorbereiteter Projektbasis etwa 30 Minuten, einschließlich fünf Minuten Projektprüfung. Erstinstallation und Dual-Core-Bootfehlersuche sind Vorbereitung der Lehrperson. Der Terminplan der Vorlesung umfasst zusammen mit Erklärung, Quiz und Abschluss 180 Minuten.

Hardware ist das **NUCLEO-H755ZI-Q mit STM32H755ZIT6**. Konfiguration und Generierung erfolgen in **STM32CubeMX**, Build, Flash und Debug in **VS Code mit STM32 VS Code Extension**. Cortex-M7 bleibt Bare Metal, Cortex-M4 verwendet den bestehenden FreeRTOS-Kernel. Die Dateien in `code/lab07/` sind ein vollständiges CM7-Anwendungsmodul, kein vollständiges Firmwareprojekt. BlueBox und winIDEA sind ein optionaler zusätzlicher Traceweg.

Teil A: Papieraufgaben

Aufgabe 1: Ausgabe und Beobachtereffekt · 8 Punkte

Eine Funktion erledigt in einem vereinfachten Modell 10 µs eigene Arbeit. Danach gibt sie die fünf ASCII-Zeichen `Hallo` bei 115200 bit/s über UART aus: ein Startbit, acht Datenbits, keine Parität, ein Stopbit je Byte. Kein Nullterminator oder Zeilenumbruch wird übertragen. Relative Abschlussfrist ab Funktionsbeginn: 50 µs.

1. Berechnen Sie Leitungsbitzahl und reine Leitungsdauer. (2 P)
2. Im **blockierenden Lehrfall** liegt die volle Leitungsdauer zusätzlich innerhalb des Antwortintervalls, weitere Kosten sind null. Berechnen Sie Antwortzeit und Fristabweichung. (2 P)
3. Warum wird dieselbe Leitungsdauer bei IRQ- oder DMA-Ausgabe nicht automatisch zur gleichen CPU-Zeit? Welche Kosten oder Wartephasen können trotzdem verbleiben? (2 P)
4. Definieren Sie einen GPIO-Messvertrag um die ursprüngliche Funktion. Nennen Sie zwei Gründe, warum ein längerer Puls keine reine Funktions-CPU-Zeit belegt, und eine geeignete Position für die Ausgabe. (2 P)

Aufgabe 2: Jobspur und Interruptkette · 8 Punkte

Jobmodell: A wird bei 1 ms freigegeben, läuft 2–4 ms und 6–7 ms und ist bei 7 ms fertig. Von 4–6 ms erledigt der Prozessor fremde Arbeit. Absolute Deadline: 8 ms. Kontextwechselkosten sind in diesem Modell null.

1. Berechnen Sie eigene CPU-Zeit, Antwortzeit, Startwartezeit und Slack. Wird die Frist gehalten? Warum ist „fertig“ nicht automatisch das Ende der Tasklebensdauer? (4 P)

Interruptmodell, gemeinsame hypothetische Uhr:

Ereignis	Zeit in µs
externe Flanke	100
ISR-Eintritt	106
Veröffentlichung	110
Task Ready	111
ISR-Ende	115
Taskstart	120
geforderte Reaktion	128

2. Berechnen Sie Eintrittslatenz, ISR-Intervall, Ready-Wartezeit und Gesamtreaktion. Geben Sie eine disjunkte Zerlegung der Gesamtreaktion an und begründen Sie, warum die vier Kennzahlen nicht miteinander aufsummiert werden. (4 P)

Aufgabe 3: Lokaler Zähler und Aussagegrenze · 8 Punkte

DWT-CYCCNT ist hier ein vorzeichenloser 32-Bit-Zähler eines Kerns. Seine Unterstützung und sein Lauf seien für das Modell bestätigt.

1. Anfang `0xFFFFFFF0`, Ende `0x00000030`: Berechnen Sie die vorzeichenlose Modulo-Differenz. Bei hypothetisch stabilen 400 MHz: Welche Zeit entspricht ihr und wie lang ist ein voller Umlauf? Warum genügt die Differenz allein nicht für ein beliebig langes Intervall? (3 P)
2. Tausend Takte werden einmal bei 400 MHz und einmal bei 200 MHz gezählt. Rechnen Sie beide Zeiten. Darf man den Endstand des M4 vom Anfangsstand des M7 subtrahieren? Welche Voraussetzungen fehlen? (2 P)
3. Eine Stichprobe vollständig definierter Intervalle enthält 12, 15, 14 und 18 µs. Berechnen Sie Anzahl, Minimum, beobachtetes Maximum und Mittelwert. Warum ist das Maximum keine WCET? Warum kann ein CYCCNT-Fenster um eine Funktion auch fremde Arbeit enthalten? (3 P)

Aufgabe 4: Quelle, Kapazität und Datenqualität · 8 Punkte

1. Ein konfiguriertes DWT-Ereignis zeigt einen M7-Store mit eins. Der Speicherbereich ist tatsächlich Write-Back-cachebar, M7 hat eine Dirty-Kopie, SRAM kann noch null enthalten. Beweist das Ereignis den vom M4 gelesenen Wert? Was muss bei einem Filter auf `ipc.payload` zusätzlich geprüft werden? Das Layout aus Vorlesung 6 hat Offsets 0/4/8. (2 P)
2. Während eines M4-Interrupts zeigt `pxCurrentTCB` weiter auf Task A. Läuft damit A-Code? Unterscheiden Sie Taskliste, Running-Profil und Zustandshistorie. (2 P)

3. Synthetisches Ratenmodell: 250000 Ereignisse/s mit vier Byte je Ereignis, Abfluss 600000 Byte/s, anfangs freier Puffer 64 KiB. Kein Paketoverhead, konstante Raten. Wann ist der Puffer voll? Würde doppelte Kapazität die dauerhafte Überlast lösen? (2 P)
4. Im Trace fehlt ein Anfangspaket. Später enthält ein Abschnitt eine Verlustmarke und danach bestätigte neue Synchronisation mit passendem Kontext. Darf der Anfang als null gezeichnet und die Lücke interpoliert werden? Wie behandeln Sie den späteren vollständigen Abschnitt? (2 P)

Teil B: lokaler Zykluszählerversuch

Aufgabe 5: die vorbereitete Projektbasis prüfen

1. Öffnen Sie die tatsächliche .ioc-Datei in STM32CubeMX und die beiden CMake-Ziele in VS Code mit STM32 VS Code Extension. Board und Bauteil entsprechen NUCLEO-H755ZI-Q und STM32H755ZIT6. Notieren Sie CubeH7-, HAL-, CMSIS-, FreeRTOS-, Compiler- und Erweiterungsversionen sowie Optimierung und beide ELF-Dateien.
2. Übernehmen Sie den bereits geprüften Boot-/MPU-/Shared-Memory-Vertrag aus Vorlesung 6. Cortex-M7 bleibt privilegiertes Bare Metal, Cortex-M4 verwendet seinen bestehenden Kernel. Ein vorheriger Ping-Pong-Basislauf wird abgeschlossen; die neue Referenz misst nicht nachträglich dessen Roundtrip und nullt seinen Block nicht. Zusätzliche Schreiber und unkontrollierte Ausgabe bleiben außerhalb des Versuchs.
3. Bestätigen Sie den tatsächlichen M7-Takt über Clock-Tree und Register. Aktualisieren Sie `SystemCoreClock` mit `SystemCoreClockUpdate()`. Ein Spezifikationsmaximum oder eine alleinige Hostanzeige ist kein tatsächlicher Labortakt. Während der Aufnahme sind Taktwechsel, Debughalt und Low-Power-/Sleepwechsel ausgeschlossen; normale Interruptarbeit wird dokumentiert.
4. Prüfen Sie im eingebundenen Cortex-M7-CMSIS-Header den DWT-Registertyp einschließlich CYCCNT, CTRL, LAR und LSR. Der Inithelper benutzt TRCENA, CYCCNTENA und die M7-Softwarelock-Konvention. Reservieren Sie die Zählerzuständigkeit mit dem Debugger: keine fremde Rücksetzung während der Aufnahme. Der Helper setzt CYCCNT nicht null und verändert keine Tracekomparatoren.
5. Die Zeitauswertung setzt ein **Intervall kürzer als einen vollen Zählerumlauf** voraus. Die Modulo-Differenz kann zusätzliche volle Umläufe nicht erkennen. Bestätigen Sie vor der Zeitinterpretation den kurzen Arbeitsfall, fortlaufenden Zähler und stabilen Takt, gegebenenfalls mit separater geeigneter Beobachtung. Ist das nicht belegt, geben Sie nur `cycles_mod32` und die offene Annahme an, keine vermeintlich sichere Mikrosekundenzeit.

Aufgabe 6: das vollständige CM7-Modul integrieren

Datei	Verwendung
`code/lab07/lab07_timing.h`	gemeinsame Deklaration innerhalb CM7
`code/lab07/lab07_timing.c`	genau einmal im CM7-CMake-Ziel

Die Quellen verwenden die Kernkennung `CORE_CM7`. Sie gehören nicht ins CM4-Ziel. Alle privaten Diagnosevariablen liegen außerhalb der reservierten SRAM4-Shared-Region; keine zweite Definition im Header anlegen. Der vorbereitete Firmwarestart wird nicht durch dieses Modul ersetzt.

Fügen Sie C-Datei und Include-Verzeichnis dem tatsächlichen CM7-Build hinzu. Im M7-Hauptkontext, nach dem geprüften Takt-, Boot- und MPU-/Cache-Start und gegebenenfalls nach dem endlichen Basis-Ping-Pong:

```
SystemCoreClockUpdate();
if (Lab07_Init(SystemCoreClock) != LAB07_OK) {
    Error_Handler();
}
if (Lab07_Run() != LAB07_OK) {
    Error_Handler();
}
```

`Error_Handler` ist der vorhandene Fehlerpfad Ihrer Projektbasis. Rufen Sie die Funktionen einmal auf, nicht bei jedem Hauptschleifenumlauf. Kein zusätzliches `osKernelInitialize` oder `osKernelStart`. Der M4-Code bleibt eigenständig. Bauen und flashen Sie die richtigen beiden Images mit der installierten STM32-Erweiterung. Prüfen Sie nach CubeMX-Regenerierung erneut die Quellenliste, Benutzerbereiche, Linkerwahl und Include-Pfade.

Was die Referenz wirklich misst: Nach einer NOP-Laufprobe erfasst sie sieben Leerfenster und sieben Arbeitsfenster. Zwischen den Zählerlesungen liegen auch DMB/DSB-Barrieren. Work verwendet eine synthetische private volatile-32-Bit-Größe für 10000 Rotier-/Additionsschritte; diese Zugriffe gehören bewusst zur gemessenen Arbeit. Das Ergebnis wird als Digest nach dem Fenster verwendet. Erst nach der Endlesung speichert das Modul den privaten Datensatz. Kein printf, keine Kernelaufrufe, kein Heap und keine Shared-Payload im Fenster.

Das ist ein **lokales Intervall**, keine reine Job-CPU-Zeit, kein IPC-Roundtrip und kein Kontrollfluss-Trace. Die Startanfrage des alten `Lab06_CM7_Run` liegt bereits vor dessen Aufruf. Auch ein Zählerfenster nur um diesen Aufruf würde nicht automatisch die vollständige erste Anforderung begrenzen.

Aufgabe 7: Diagnose auswerten und Bedingungen berichten

Nach beendetem Logging lesen Sie die private Diagnose mit dem richtigen CM7-Image und einem zum M7-Cache passenden Debugzugriff. Eine rohe SRAM- oder Live-Memory-Ansicht ist nicht automatisch die vom laufenden Kern benutzte Sicht. Halt während der Aufnahme würde deren Bedingungen verändern.

Feld	Logisches Soll / Interpretation
`lab07_status`	LAB07_OK im gültigen Lauf
`lab07_count`, `lab07_done`	14, 1
Sequenz / Art	1–7 EMPTY, 8–14 WORK
`lab07_dropped`	null im unveränderten 14-Einträge-Lauf
`core_hz`	bestätigte M7-Frequenz
`digest`	EMPTY null, WORK gleiches rechnerisches Resultat
`cycles_mod32`	tatsächlich beobachteter 32-Bit-Restwert

1. Ordnen Sie die sieben Werte je Gruppe zu. Berichten Sie Anzahl, Minimum und beobachtetes Maximum, zunächst in Taktresten. Rechnen Sie nur bei gültigen Zähler-/Takt-/Umlaufbedingungen mit $t = \Delta c / f_{Kern}$ um.
2. Berichten Sie den Leerwert separat und erklären Sie, welche Kosten zum Fenster gehören. Subtrahieren Sie nicht unbemerkt einen fremden Minimalwert von jedem Arbeitswert und behaupten dann eine reine WCET.

3. Dokumentieren Sie die beobachtete Streuung, tatsächlichen Takt, Optimierung, Cache/MPU, Last, Interruptbedingungen und Debugzustand. Ein gleiches Digest belegt dasselbe berechnete Ergebnis, keine gleiche Laufzeit und keine vollständige Fristgarantie.
4. Der Logvertrag hat sechzehn Plätze und einen Schreiber. Als optionale **Kapazitätsprüfung nach dem abgeschlossenen Basislauf** dürfen Sie im selben M7-Hauptkontext zwei weitere `Lab07_RecordOne(LAB07_EMPTY)`-Aufrufe ausführen. Ein dritter Zusatzaufruf meldet `LAB07_FULL`, erhöht den Dropzähler und überschreibt die ersten sechzehn Datensätze nicht. Dies sind zusätzliche Versuchsaufrufe, kein Fehler des unveränderten 14-Einträge-Laufs. Keine parallelen ISR- oder Threadschreiber hinzufügen.
5. Wenn Initialisierung oder Build scheitert, dokumentieren Sie den tatsächlichen technischen Status. Ein nicht durchgeführter Boardlauf bleibt als solcher gekennzeichnet. Mikrosekundenfelder ohne gültige Beobachtung bleiben leer.

Optional: vorhandenen Hardwaretrace am bekannten Kanal prüfen

Nur mit tatsächlich bestätigtem Gerät und Aufnahmepfad. Führen Sie nicht mehrere unbestätigte Debugserver parallel. Das Werkzeug dient hier zur zusätzlichen Aufnahme; die Firmwareentwicklung bleibt CubeMX und VS Code mit STM32-Erweiterung.

1. Notieren Sie Modell, Revision, Probe/Adapter, Pins, winIDEA-Version, unterstützte Quellen und Zeitbasis. Ein SWD-Debugzugang beweist kein SWO, keinen Paralleltrace und keine gemeinsame Uhr. Anschlussablauf und Pegel richten sich nach den tatsächlichen Geräte- und Boardunterlagen.
2. Ordnen Sie je Kern die geflashte ELF zu. Prüfen Sie zunächst einen bekannten Zugriff mit Kern, Adresse, Breite und Richtung. Fehlendes erstes Paket ist nicht null.
3. Beobachten Sie den unveränderten nicht cachebaren Kanal aus Vorlesung 6: Antworten 1, 3, 5, Endpayload fünf und TURN_M7. Payload liegt bei Offset acht, state bei vier. Bestätigen Sie die tatsächlich erfasste Quelle statt einen DWT-Zugriff als vollständige SRAM-Historie auszugeben.
4. Inter-Core-Abstände benötigen gleichzeitig erfasste Quellen und bestätigte Zeitkorrelation. Zwei getrennte Zähler oder Aufnahmen genügen nicht. M4-Taskzuordnung während einer ISR ist keine Task-CPU-Zeit. Zustandsereignisse und Anforderungsmarken sind zusätzliche Information.
5. Ein cachebarer Fehlerbuild ist ein eigener gemeinsamer Neustart mit nachgewiesenen effektiven Attributen und laufendem D-Cache. Alte Werte oder Timeout sind mögliche Beobachtungen, keine vorgeschriebene Kurve. Kein Nullen des möglicherweise noch benutzten Blocks. Nach dem Vergleich stellen Sie die nicht cachebare Basis wieder her.

Ohne solche Hardware wird die Interpretation mit den Papierfällen bearbeitet und die Aufnahme als nicht durchgeführt protokolliert. Eine erfüllte Datenfolge und eine aufgezeichnete Zeitfrist sind verschiedene Nachweise.

Abgabe und Quellen

Teil A: vier begründete Papierlösungen, 32 Punkte. Teil B: tatsächliche Projektversionen und Takte, Quellen-/Regenerierungsprüfung, Bedingungen des Zählers, Diagnose pro Gruppe und ihre Aussagegrenzen. Trennen Sie erwartet, beobachtet, Modellrechnung und nicht durchgeführt. Optionaler Trace erhält ein eigenes Aufnahmeprotokoll.

Quellen: Vorlesung 7; Giorgio C. Buttazzo, *Hard Real-Time Computing Systems*, 3. Auflage; ST RM0399, UM2408 und AN4989; tatsächliche CMSIS-M7-Header, insbesondere DWT-Register und

ITM_SendChar; Dokumentation der installierten STM32 VS Code Extension und gegebenenfalls des vorhandenen winIDEA-/BlueBox-Aufbaus.